Лабораторная работа №7. Коллективные операции в MPI

Поздняков Илья, группа 11-401

1. Что делает программа

mpi_array_sum.c вычисляет сумму массива $a_i = i + 1$, $i = 0 \ldots N-1$, и сверяет её с $N(N+1)/2$. **MPI_Bcast**: процесс 0 читает N из аргумента командной строки и рассылает его всем; если $N \le 0$, все процессы получают это значение и одинаково завершаются через MPI_Finalize, поэтому зависаний нет. **MPI_Scatter**: процесс 0 формирует массив и раздаёт каждому по N/P элементов (доп. задание: если N не делится на P, используется **MPI_Scatterv** — первые N mod P процессов получают на 1 элемент больше). Каждый процесс считает локальную сумму s_r (тип long long, MPI_LONG_LONG_INT). **MPI_Gather** собирает все s_r в массив на процессе 0, где они складываются циклом; **MPI_Reduce** (MPI_SUM) сразу даёт сумму на процессе 0; **MPI_Allreduce** даёт сумму всем процессам — доп. задание: каждый процесс по общему итогу вычисляет и печатает свою долю s_r/S, для чего итог нужен всем, и Allreduce делает это одним вызовом вместо Reduce + Bcast. Все коллективные вызовы выполняют все процессы в одинаковом порядке.

2. Сборка, запуск и методика

`mpicc -O2 -Wall -Wextra mpi_array_sum.c -o mpi_array_sum`, `mpiexec -n 4 ./mpi_array_sum 1000000 [reps]`. Стенд: Intel Xeon @ 2.10 ГГц (виртуальная машина), 2 ядра, Open MPI 4.1.6, GCC 13.3.0; для P = 4 и 8 процессов больше, чем ядер, поэтому использовался `--oversubscribe`. Время (MPI_Wtime) — Scatter + локальная сумма + Gather + Reduce + Allreduce; массив формируется до замера. Одна пробная итерация, затем 50 повторений, берётся максимум по процессам и среднее; каждая конфигурация запущена 5 раз, в таблицах среднее по запускам. Время одной операции агрегирования — 10^5 повторений подряд.

3. Результаты

Табл. 1. Корректность (3.1)

P	N	Ожидаемая сумма	Gather	Reduce	Allreduce	Корректно?
2	100	5 050	5 050	5 050	5 050	да
4	100	5 050	5 050	5 050	5 050	да
4	10 000	50 005 000	50 005 000	50 005 000	50 005 000	да
8	1 000 000	500 000 500 000	500 000 500 000	500 000 500 000	500 000 500 000	да

Табл. 2. Размер массива (3.2)

P	N	Время, с	Верно?
4	10 000	$2.26 \cdot 10^{-5}$	да
4	100 000	$5.40 \cdot 10^{-5}$	да
4	1 000 000	$5.31 \cdot 10^{-4}$	да
4	10 000 000	$6.42 \cdot 10^{-3}$	да

Табл. 3. Число процессов (3.3), база — P = 1

N	P	Время, с	S_p	E_p
1 000 000	1	$6.51 \cdot 10^{-4}$	1.00	1.00
1 000 000	2	$5.57 \cdot 10^{-4}$	1.17	0.58
1 000 000	4	$5.31 \cdot 10^{-4}$	1.23	0.31
1 000 000	8	$5.67 \cdot 10^{-4}$	1.15	0.14

Табл. 4. Gather + сумма против Reduce (3.4)

P	Способ	Повторений	Среднее время операции, с	Верно?
2	Gather + сумма на 0	$10^5 \times 5$	$1.10 \cdot 10^{-7}$	да
2	Reduce	$10^5 \times 5$	$1.53 \cdot 10^{-7}$	да
4	Gather + сумма на 0	$10^5 \times 20$	$7.82 \cdot 10^{-7}$	да
4	Reduce	$10^5 \times 20$	$6.94 \cdot 10^{-7}$	да
8	Gather + сумма на 0	$10^5 \times 5$	$2.02 \cdot 10^{-6}$	да
8	Reduce	$10^5 \times 5$	$1.39 \cdot 10^{-6}$	да

4. Вывод

Все три способа дали одинаковую и верную сумму во всех конфигурациях, в том числе 500 000 500 000 при $N = 10^6$, что требует 64-битного типа. Время растёт примерно пропорционально N (×10 по N — ×10 по времени при $N \ge 10^5$): основная доля — раздача массива через Scatter. Ускорение от числа процессов мало ($S \approx 1{,}2$ при P = 2–4): локальная сумма очень дешёвая, а Scatter — это последовательное копирование всего массива из памяти процесса 0, которое не ускоряется; к тому же на машине 2 ядра, и при P = 4, 8 процессы делят ядра — эффективность падает до 0,14. Это иллюстрирует вывод лекции: при малом объёме вычислений коммуникации ограничивают ускорение. **Gather, Reduce и Allreduce различаются тем, что и где получается:** Gather собирает отдельные значения в массив на корневом процессе (суммирование пишем сами, но можно увидеть вклад каждого процесса), Reduce сразу объединяет значения операцией (MPI_SUM) и оставляет один результат на root — код короче и MPI может объединять по дереву, Allreduce даёт результат всем процессам. По времени для одного числа обе операции занимают от 0,1 до 2 мкс: при P = 2 чуть быстрее Gather, при P = 4 и 8 — Reduce (на 10–30 %), и с ростом P время обеих растёт. Разница мала, поэтому выбор определяется задачей: нужен результат только на 0 — Reduce, всем процессам — Allreduce.